Unit 3: Input/Output Functions — Summary Notes

1. Overview of I/O in C

- **No Built-in Keywords: C has no built-in language keywords for receiving input or displaying output. Instead, all I/O operations are performed using standard library functions provided in standard header files (such as <stdio.h> and <conio.h>).**
- **Console I/O vs. File I/O:**
    - **Console I/O: Refers to operations involving the keyboard (standard input stdin) and the screen/VDU (standard output stdout).**
    - **File I/O: Refers to reading and writing data stored on secondary storage media (e.g., hard drives).**

---

2. Categorization of Console I/O Functions

Console I/O functions are divided into Formatted and Unformatted functions:

Feature	Formatted I/O Functions	Unformatted I/O Functions
Definition	Allows input and output to be formatted (field width, alignment, decimal places).	Processes raw characters or strings without formatting overhead.
Input Functions	scanf(), sscanf()	getch(), getche(), getchar(), gets()
Output Functions	printf(), sprintf()	putch(), putchar(), puts()

---

3. Formatted I/O Functions

A. printf() (Formatted Output)

- **Purpose: Outputs formatted data onto the screen.**
- **Syntax: printf("format string", list_of_variables);.**
- **Execution: Evaluates the format string from left to right. Literal text is printed directly; conversion specifications (starting with %) pick corresponding variables, and escape sequences (starting with \) execute formatting actions.**

B. scanf() (Formatted Input)

- **Purpose: Receives formatted input from the keyboard.**
- **Syntax: scanf("format string", list_of_addresses);.**

- **Address Operator (&): Requires the ampersand & before non-pointer variable names to supply memory addresses where input values will be stored.**
- **Delimiters: Input values typed at the keyboard must be separated by spaces, tabs, or newlines.**
- **Limitation: scanf() stops reading a string at the first whitespace, making it unsuitable for multi-word string input.**

C. In-Memory Formatted Functions

- **sprintf(): Formats data and writes the result into a character array/string buffer instead of displaying it on the screen.**
- **sscanf(): Reads formatted data directly from a string buffer in memory instead of reading from the keyboard.**

---

4. Conversion Specifiers (Format Specifiers)

Conversion specifiers inform printf() and scanf() about the data type being processed:

- **Integers:**
  - **%d or %i: Signed decimal integer**
  - **%u: Unsigned decimal integer**
  - **%ld / %lu: Long signed / long unsigned integer**
  - **%o: Unsigned octal integer**
  - **%x: Unsigned hexadecimal integer**
- **Floating-Point Numbers:**
  - **%f: Floating-point (float)**
  - **%lf: Double-precision float (double)**
  - **%Lf: Long double (long double)**
- **Characters & Strings:**
  - **%c: Single character (char)**
  - **%s: Character string (char[] or char *)**

Optional Format Flags & Modifiers

- **Field Width (%dd): Specifies the minimum number of screen columns reserved for output (e.g., %10d reserves 10 columns, right-justified).**
- **Left Justification (-): Left-aligns the output within the allocated field width (e.g., %-10d).**

- **Precision (.dd): Specifies the number of decimal places for floating-point values (e.g., %10.2f prints 2 decimal places).**

---

5. Unformatted I/O Functions

A. Single Character Functions

- **Input:**
  - **getch(): Reads a character from the keyboard immediately without echoing it on screen and without waiting for the Enter key.**
  - **getche(): Reads a character immediately, echoes it on screen, and does not require pressing Enter.**
  - **getchar() / fgetchar(): Reads a character with echo, but requires pressing the Enter key to confirm input.**
- **Output:**
  - **putch(), putchar(), fputchar(): Outputs a single character to the screen.**

B. String Functions

- **gets(): Reads a line of text (including multi-word strings with spaces) from the keyboard until the Enter key (\n) is pressed. Replaces \n with a null character \0.**
- **puts(): Displays a string on the screen and automatically appends a newline (\n) at the end.**

---

6. Common Escape Sequences

Escape sequences begin with a backslash \ and alter standard character interpretation:

- **\n: Newline — moves the cursor to the beginning of the next line.**
- **\t: Horizontal Tab — advances the cursor to the next tab stop.**
- **\b: Backspace — moves the cursor one position to the left.**
- **\r: Carriage Return — moves the cursor to the start of the current line.**
- **\a: Alert / Bell — produces a beep sound.**
- **\', \", \\: Prints a literal single quote, double quote, or backslash.**